1 What we have

A setting of learning is an *ideal functionality*, a *scenario* is an ideal functionality with a coalition and the coalition's behavior, and an *attack* is a question asked in a scenario. The possible attacks come from the two sides of an ideal functionality that face the coalition, what it hands out and what it accepts, so they are generated from the settings and not collected from papers. Ten standard attacks result, in three question kinds, and the library holds ten ideal functionalities, among them FedAvg in three variants, split and vertical learning, private inference and HE-OFT. Two ideal functionalities with the same parties are ordered by what they hand a coalition, and an attack's advantage can only grow along that order (proven). Each attack is therefore located by its lowest ideal functionalities, so the relation between attacks and ideal functionalities is a family of up-sets and not an arbitrary many-to-many map. The lowest ideal functionalities of an attack say where encryption must go to stop it, and the floor of a task, the lowest ideal functionality that still computes it, holds the attacks no encryption stops. How much an attack gains where it runs is a separate quantitative question, answered by a bound or by a measurement.

Does not show: The definitions are ours, written in the simulation paradigm that secure aggregation is already proven in, and no other reader has checked them. An empirical verdict rests on the ledger's coding of the paper, and of the works cited only Yeom et al., Salem et al. and HE-OFT were read in full.

2 Scenarios

The parties are a server, clients $1, \dots, n$ and a querier, and each party P holds an input x_P. Client i holds a dataset D_i, a multiset of records, the server a model or the messages it chooses, and the querier a query. For a set B of parties, x_B collects their inputs.

An ideal functionality $\mathcal{F}$ is one incorruptible party that receives every input and hands each party P an output $\mathcal{F}_P(x)$. It also fixes an *input space* for each party, the set of inputs it accepts from that party. A setting that runs in rounds takes a fresh input each round and hands out a sequence of outputs, and $\mathcal{F}_P(x)$ then denotes the whole sequence. The outputs are the side the coalition observes and the input spaces the side through which it acts, and the library below writes both for every setting.

A *coalition* A is the set of parties the adversary controls, and H is the set of the honest parties. The coalition's *view* under $\mathcal{F}$ is $(x_A, \mathcal{F}_A(x))$, its own inputs and every output handed to one of its members. Its *behavior* is one of three.

- A *semi-honest* coalition supplies its true inputs and only observes.
- An *input-choosing* coalition supplies any inputs its input spaces allow, chosen from what it has seen so far, and aborts where $\mathcal{F}$ permits it. A protocol secure against malicious coalitions leaves the adversary exactly this power.
- A *deviating* coalition departs from the protocol. Against a protocol secure only for semi-honest coalitions the simulation then says nothing, so such an attack is placed on the protocol and not on $\mathcal{F}$.

A scenario is the triple $(\mathcal{F}, A, b)$ of an ideal functionality, a coalition and a behavior b.

A protocol π *realizes* $\mathcal{F}$ against A with error ϵ when a simulator, given only $(x_A, \mathcal{F}_A(x))$, produces the coalition's view in π up to distance ϵ. The distance is taken jointly with the honest outputs and holds for every input x, which is the definition secure aggregation is proven against [10, 9]. Every question below then gains at most 2ϵ more in π than under $\mathcal{F}$ (proven, since each hypothesis moves the coalition's view by at most ϵ). A question therefore splits into a cryptographic layer, the size of ϵ, and an output layer, the advantage under $\mathcal{F}$, and the rest of this report concerns the output layer. Threshold CKKS computes the outputs only approximately, so it realizes a variant of

$\mathcal{F}$ whose outputs carry the CKKS error, and its realization error rests on a smudging hypothesis (heuristic, `lib/reports/2026-11-11-halil-he-results-carried-over.pdf`).

3 Questions and the standard attacks

A question is posed on the honest inputs x_H and the honest outputs, and it comes in one of three kinds.

- In a *distinguish* question two distributions $\mathcal{D}_0$ and $\mathcal{D}_1$ over x_H are fixed, x_H is drawn from $\mathcal{D}_\beta$ for a hidden bit β, and the coalition guesses β from its view. Its advantage is $|\Pr[\text{guess } 1 \mid \beta = 1] - \Pr[\text{guess } 1 \mid \beta = 0]|$.
- In an *invert* question a target function g, a loss ℓ and a tolerance τ are fixed, and the coalition wins when its output y satisfies $\ell(y, g(x_H)) \leq \tau$. Its advantage is its probability of winning minus that of the best guess made from the prior on x_H alone.
- In a *steer* question a target set T of honest outputs is fixed, and an input-choosing coalition picks inputs x'_A from its input spaces. Its advantage is $\Pr[\mathcal{F}_H(x'_A, x_H) \in T]$ minus the same probability under its true inputs.

Distinguish and invert questions act through what $\mathcal{F}$ hands out and steer questions through what it accepts, so a semi-honest coalition has zero steering advantage by definition.

A standard attack is one question kind with one target that the input and output types of the library admit, and Table 1 lists the ten. The names are targets, so membership inference in a paper is the membership question asked in that paper's scenario. Source keeps the union and every dataset size fixed, because an ideal functionality that hands out the sizes would otherwise answer it outright. By construction every honest input type has an invert target and every honest output a steer target. Whether the list covers every attack of interest is open, and free-riding is a known case outside it, since it concerns what the coalition contributes.

Table 1. The standard attacks. Each is one question kind with one target, and the interface is the side of the ideal functionality it acts through.

Name	Kind	The two hypotheses, or the target	Interface
membership	distinguish	a record z is in an honest dataset, or it is not	outputs
source	distinguish	z is with honest client i and z' with honest client j, or the two are swapped, so the union and every size stay fixed	outputs
property	distinguish	the honest data come from a distribution with a property, or without it	outputs
record	invert	a record of an honest dataset, within a distance	outputs
attribute	invert	one feature or the label of a record whose other features are known	outputs
model	invert	the parameters or the behavior of a model that is an honest input	outputs
query	invert	the query of an honest querier	outputs
targeted	steer	honest outputs whose model maps chosen inputs to chosen labels	inputs
untargeted	steer	honest outputs whose model has accuracy below a level	inputs
delivery	steer	the honest parties receive no output	inputs

4 The library of ideal functionalities

Table 2 writes each setting as an ideal functionality, with what it accepts and what it hands to the server, to a client and to the querier. Variants of one setting are different ideal functionalities, and fedavg-plain, fedavg-secagg and fedavg-final differ only in what the server receives. The server's

input in fedavg-plain and fedavg-secagg is the model it sends each client each round, because deployed protocols let the server choose it. Requiring one model for all clients gives a variant with a smaller input space, which a protocol realizes only by checking that every client received the same model. The clients' local training is part of $\mathcal{F}$, so an honest client gives its dataset and $\mathcal{F}$ computes its update, while an input-choosing client may submit any update. In he-oft the head displacement and the per-class counts are computed in the same way, and the server's output is the leakage of the simulation theorem of HE-OFT [7]. The ideal functionality outsourced-training has a single data owner, so the framework also covers settings that are not multi-party.

Table 2. The library of ideal functionalities. A client that chooses its input may submit anything the Accepts column allows, and every output not listed is withheld.

Name	Accepts	The server receives	A client receives	The querier receives
pooled-training	a dataset from each client	the model trained on the shuffled union	the same model	nothing
served-model	a dataset from each client, queries	nothing	nothing	the trained model's answer to each query
outsourced-training	a dataset from one client	nothing	the model trained on it	nothing
fedavg-plain	a dataset or any update from each client, a model for each client each round from the server	every client's update in every round	the model sent to it	nothing
fedavg-secagg	as fedavg-plain	the sum of the updates in every round	the model sent to it	nothing
fedavg-final	a dataset or any update from each client	the final model	the average model of every round	nothing
split-learning	batches from the client, any gradient at the cut from the server	the activations at the cut and the labels	the gradient at the cut	nothing
vertical-training	a feature block of the shared records from each client, the labels and any gradient from the server	every client's embedding	the gradient at its own embedding	nothing
private-inference	a model from the server, a query	nothing	nothing	the model's answer to the query
he-oft	a dataset, or any head displacement and per-class counts, from each client, up to Q queries [7]	each client's sample count and the selected arrangement	the selected arrangement	one label per query

5 The order, and where an attack runs

Fix a coalition A and two ideal functionalities $\mathcal{F}$ and $\mathcal{G}$ with the same parties and the same honest input spaces. $\mathcal{F}$ is *below* $\mathcal{G}$ for A, written $\mathcal{F} \preceq_A \mathcal{G}$, when an efficient map m satisfies

$$(x_A, \mathcal{F}_A(x)) \stackrel{d}{=} m(x_A, \mathcal{G}_A(x)) \quad \text{for every input } x,$$

so that the coalition computes its view under $\mathcal{F}$ from its view under $\mathcal{G}$. If $\mathcal{F} \preceq_A \mathcal{G}$, a semi-honest coalition gains no more under $\mathcal{F}$ than under $\mathcal{G}$ in any distinguish or invert question (proven, since a

strategy under $\mathcal{F}$ runs under $\mathcal{G}$ after m). For a level δ, the ideal functionalities on which an attack's advantage exceeds δ thus form an up-set, and the attack is located by the minimal elements of that set. Steering is ordered by the input spaces instead, because an ideal functionality that computes the same honest outputs and accepts fewer inputs from A admits no more steering (proven, as a maximum over a smaller set).

For a semi-honest server, fedavg-final $\preceq$ fedavg-secagg $\preceq$ fedavg-plain (proven), because the server computes the final model from the round sums and each round sum from the updates. The ideal functionality pooled-training is comparable with none of the three, because its output does not depend on how the records are split among the clients and theirs does.

An input-choosing server acts through both sides at once and can move itself up the order. Pasquini et al. [14] send a crafted model to every client but the target, so the round sum under fedavg-secagg equals the target's update, which is what fedavg-plain hands out (empirical). The variant with one model for all clients blocks that route, yet Shi et al. [18] still reconstruct the records of the union with one crafted model, without attributing them to clients (empirical). In the terms of Table 1, the variant still admits record, and that attack gives no answer to source. The order therefore locates an active attack only once the input space is fixed, so the input space is a coordinate of an ideal functionality as much as its outputs are.

6 Where encryption goes, and the floor

A protocol stops an attack in a scenario when the ideal functionality it realizes lies outside the attack's up-set. Encryption and MPC let a protocol realize a lower ideal functionality of the same task, fedavg-secagg in place of fedavg-plain for instance, and each step down costs computation. Placing HE is then the choice of the highest ideal functionality of the task, the one with the least encryption, that lies outside the up-set of every attack the deployment must stop.

A task fixes what some party must receive, such as the trained model for the clients or one label per query for the querier. Its *floor* is the lowest ideal functionality of the library that still hands that out, and an attack whose up-set contains the floor runs under every protocol for the task. Encryption cannot stop such an attack, and what remains is to change the ideal functionality, by DP inside it or by narrowing what it accepts, or to record the leakage as inherent. Each attack in a scenario thus gets one of three verdicts at the output layer: zero advantage, where encryption alone suffices, a bound from DP inside the ideal functionality, or no bound, where a measurement or a gap is recorded. Table 3 gives worked cases.

Source has zero advantage under fedavg-secagg with one full-batch local step weighted by size, because client i's update at round t is the sum of the per-record gradients of D_i, so the round sum is

$$\sum_{i=1}^{n} \sum_{z \in D_i} \nabla L(\theta_t, z) \;=\; \sum_{z \in D_1 \uplus \cdots \uplus D_n} \nabla L(\theta_t, z),$$

a function of the union (proven), where θ_t is the model of round t, L the training loss and $\uplus$ the union of multisets. Several local steps break the identity, because the update is then a nonlinear function of the client's own records, so the round sum depends on the split.

HE-OFT shows the floor of a task that serves labels. Its shared displacement for class c is the count-weighted mean of the clients' displacements $\Delta_{j,c}$ with per-class counts $g_{j,c}$, so a client a that inflates its count moves it at will,

$$\frac{\sum_j g_{j,c}\, \Delta_{j,c}}{\sum_j g_{j,c}} \;\longrightarrow\; \Delta_{a,c} \qquad \text{as } g_{a,c} \to \infty$$

(proven, prop-he-oft-count-steering of the carried-over report). The counts are inputs he-oft accepts, so encryption leaves this steering in place, and only a variant that bounds the accepted counts removes it.

Table 3. Worked cases. The verdict is about the ideal functionality, so a protocol realizing it adds at most its realization error. The active attacks on fedavg-secagg are discussed in the text.

Attack	Scenario	Verdict at the output layer	Status
source	pooled-training, any coalition holding neither z nor z'	zero, because every output is a function of the union and the coalition's inputs	proven
source	fedavg-secagg or fedavg-final, semi-honest server, one full-batch local step weighted by size	zero, because every round's sum is the sum of the per-record gradients over the union	proven
source	the same, with several local steps	the model depends on how the records are split, so the advantage need not vanish, and its size is unknown	open
source	fedavg-plain, semi-honest server	at least the advantage of telling whether one client's update was computed with z or with z', since the server reads that update alone	proven
membership	every training task that hands out a model or answers	in the floor, and at most $e^{\varepsilon} - 1$ when the ideal functionality trains with ε-DP [22]	proven
targeted, untargeted	pooled-training, input-choosing client	in the floor, because the ideal functionality accepts any dataset	proven
targeted	he-oft, input-choosing client	in its floor, because inflated per-class counts move a row of the shared head at will [7]	proven
membership	he-oft, querier with Q queries	in its floor, and at most the advantage of a holder of both heads in plaintext [7]	proven
every output question	outsourced-training, server coalition	zero, because the server receives nothing	proven

7 Published attacks, placed

The ledger codes, for each of its 103 attack threat models, what the attack reads in plaintext, and the least of it gives the lowest ideal functionality of the library under which the attack runs as published (empirical, coded on 2026-10-16 and not re-read). Of these, 40 read a client's own update and 30 a per-round sum or model, so an ideal functionality that hands the server only the final model stops these 70 as published. Another 12 read values at a cut, which places them under split-learning and vertical-training. Only 4 need no more than the final model or served answers, which puts them in the floor of every training task, and 4 read a decrypted value, which attacks the realization and not the ideal functionality. The other 13 are coded as reading nothing in plaintext or values outside the library, and the placement ignores the input space an attack uses.

8 Prior work

Table 4 lists the works this report compares against, what each fixes in the picture above, and where its PDF is held. Each fixes one piece, the vocabulary, the games on one model, the simulation of one protocol or one joint construction, and the gap they leave is the subject of `lib/reports/2026-11-11-halil-prior-work-on-scenario-frameworks.pdf`. Barreno et al. and the three standards are not held, the first behind the Springer paywall and the others because they are sold.

9 Open

- Active coalitions need an order on pairs of outputs and input spaces, and the up-set statement is proven here only for a fixed input space.
- Settings that run in rounds against adaptive coalitions need the order round by round, which

Table 4. The prior work this report compares against, what each fixes in the picture of ideal functionalities and attacks, and the folder of its PDF under lib/literature/pdfs, where the file name starts with the key.

Work	What it fixes in this picture	PDF folder
Barreno et al. [1]	goal and capability, which become the two interfaces	not held
Papernot et al. [13]	the stage of an attack and the adversary's knowledge	ieee-eurosp
NIST AI 100-2 [19]	the names of attacks, privacy and integrity together	nist
Yeom et al. [22]	membership and attribute games on one model, the DP bound	csf
Salem et al. [17]	18 games on one model, the output layer of pooled-training	ieee-sp
Li et al. [8]	games on gradients, the output layer of fedavg-plain	acm-ccs
Wu et al. [21]	integrity attacks, the steer kind, with no advantage	arxiv
Mansouri et al. [10]	the ideal functionality of secure aggregation, by coalition	popets
Ma et al. [9]	simulation of secure aggregation over many rounds	ieee-sp
Morona-Mínguez et al. [11]	averaging under multi-key HE, up to the CKKS error	arxiv
HE-OFT [7]	an ideal functionality with stated leakage, and the bound on label queries	arxiv
Pentyala et al. [15]	MPC for the protocol and DP inside the ideal functionality	arxiv
Carletti et al. [2]	the threat models of gradient inversion, by role	usenix-security
Rizzini et al. [16]	verifiable integrity, which narrows what is accepted	popets
Desfontaines and Pejó [3]	criteria for a list of attacks, exclusive and compatible dimensions	popets
NIST SP 800-226 [12]	the unit of privacy for DP inside an ideal functionality	nist
Villa et al. [20]	one game for confidentiality, integrity and availability of agentic LLM systems	arxiv
IEEE 3652.1 [4], ISO/IEC 4922 [5], ISO/IEC 28033 [6]	the vocabulary of federated learning, MPC and FHE	not held

the definition on whole output sequences does not give.

- Each ideal functionality should be stated with the noisy output that threshold CKKS computes, so that the realization error concerns the protocol alone.
- The library lacks decentralized learning, fine-tuning with LoRA adapters across parties, and inference with a model split across parties.
- Source under several local steps is open, and it decides whether the example of client A and client B holds for FedAvg as deployed.

References

[1] Marco Barreno, Blaine Nelson, Anthony D. Joseph, and J. D. Tygar. The security of machine learning. *Machine Learning*, 2010.

[2] Vincenzo Carletti, Pasquale Foggia, Carlo Mazzocca, Giuseppe Parrella, and Mario Vento. SoK: Gradient inversion attacks in federated learning. In *USENIX Security Symposium*, 2025.

[3] Damien Desfontaines and Balázs Pejó. SoK: Differential privacies. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2020.

[4] IEEE. IEEE guide for architectural framework and application of federated machine learning. IEEE Std 3652.1-2020, 2020.

[5] ISO/IEC. Information security, secure multiparty computation, part 1: General. ISO/IEC 4922-1:2023, 2023.

[6] ISO/IEC. Fully homomorphic encryption, part 1: General. ISO/IEC DIS 28033-1, draft, 2026.

[7] Halil İbrahim Kanpak, Sinem Sav, and Alptekin Küpçü. HE-OFT: Privacy-preserving one-shot federated fine-tuning under homomorphic encryption. arXiv:2610.08255, 2026.

[8] Zhuohang Li, Andrew Lowy, Jing Liu, Toshiaki Koike-Akino, Kieran Parsons, Bradley Malin, and Ye Wang. Analyzing inference privacy risks through gradients in machine learning. In *ACM Conference on Computer and Communications Security (CCS)*, 2024.

[9] Yiping Ma, Jess Woods, Sebastian Angel, Antigoni Polychroniadou, and Tal Rabin. Flamingo: Multi-round single-server secure aggregation with applications to private federated learning. In *IEEE Symposium on Security and Privacy (S&P)*, 2023.

[10] Mohamad Mansouri, Melek Önen, Wafa Ben Jaballah, and Mauro Conti. SoK: Secure aggregation based on cryptographic schemes for federated learning. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2023.

[11] Miguel Morona-Mínguez, Fernando Pérez-González, and Alberto Pedrouzo-Ulloa. Pushing forward multi-secret-key homomorphic encryption for private average aggregation. arXiv:2609.01945, 2026.

[12] Joseph P. Near and David Darais. Guidelines for evaluating differential privacy guarantees. Technical Report SP 800-226, NIST, 2025.

[13] Nicolas Papernot, Patrick McDaniel, Arunesh Sinha, and Michael P. Wellman. SoK: Security and privacy in machine learning. In *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018.

[14] Dario Pasquini, Danilo Francati, and Giuseppe Ateniese. Eluding secure aggregation in federated learning via model inconsistency. In *ACM Conference on Computer and Communications Security (CCS)*, 2022.

[15] Sikha Pentyala, Davis Railsback, Ricardo J. M. Maia, David Melanson, Rafael Dowsley, Anderson Nascimento, and Martine De Cock. Training differentially private models with secure multiparty computation. arXiv:2202.02625, 2022.

[16] Andrea Rizzini, Marco Esposito, Tommaso Gagliardoni, and Francesco Bruschi. SoK: Verifiable integrity claims for privacy-preserving federated learning. *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2026.

[17] Ahmed Salem, Giovanni Cherubin, David Evans, Boris Köpf, Andrew Paverd, Anshuman Suri, Shruti Tople, and Santiago Zanella-Béguelin. SoK: Let the privacy games begin! A unified treatment of data inference privacy in machine learning. In *IEEE Symposium on Security and Privacy (S&P)*, 2023.

[18] Shanghao Shi, Ning Wang, Yang Xiao, Chaoyu Zhang, Yi Shi, Y. Thomas Hou, and Wenjing Lou. Scale-MIA: A scalable model inversion attack against secure federated learning via latent space reconstruction. In *Network and Distributed System Security Symposium (NDSS)*, 2025.

[19] Apostol Vassilev, Alina Oprea, Alie Fordyce, Hyrum Anderson, Xander Davies, and Maia Hamin. Adversarial machine learning: A taxonomy and terminology of attacks and mitigations. Technical Report AI 100-2e2025, doi:10.6028/NIST.AI.100-2e2025, NIST, 2025.

[20] Federico Villa, F. Betül Durak, Tadayoshi Kohno, Tapdig Maharramli, and Franziska Roesner. Extending the formalism and theoretical foundations of cryptography to AI. arXiv:2603.02590, 2026.

[21] Baoyuan Wu, Zihao Zhu, Li Liu, Qingshan Liu, Zhaofeng He, and Siwei Lyu. Attacks in adversarial machine learning: A systematic survey from the life-cycle perspective. arXiv:2302.09457, 2023.

[22] Samuel Yeom, Irene Giacomelli, Matt Fredrikson, and Somesh Jha. Privacy risk in machine learning: Analyzing the connection to overfitting. In *IEEE Computer Security Foundations Symposium (CSF)*, 2018.